Modelado y gestión de bases de datos

Breve descripción:

Este componente formativo aborda los fundamentos del modelado y la gestión de bases de datos, incluyendo diseño conceptual, lógico y físico, lenguajes DDL y DML, integridad de los datos y sistemas gestores de bases de datos relacionales y NoSQL. Además, analiza herramientas de modelado, almacenamiento de datos y principios aplicados en sistemas distribuidos.

Octubre 2026

Tabla de contenido

Introducción 5

1. Fundamentos de bases de datos 8

1.1. Conceptos básicos y características de las bases de datos 8

1.2. Evolución y tendencias de las bases de datos 11

1.3. Modelos de datos estructurados, semiestructurados y desestructurados 14

1.4. Bases de datos relacionales, NoSQL y NewSQL 18

2. Modelado de bases de datos..... 24

2.1. Diseño conceptual de bases de datos 25

2.2. Modelo Entidad-Relación (ER) 27

2.3. Diseño lógico de bases de datos..... 31

2.4. Transformación del modelo conceptual al modelo lógico 33

2.5. Diseño físico de bases de datos 34

2.6. Normalización de bases de datos 36

3. Interpretación y estructura del modelo relacional 40

3.1. Entidades, atributos y relaciones..... 40

3.2. Llaves primarias y foráneas 41

3.3. Integridad de identidad e integridad referencial 43

3.4. Restricciones y consistencia de los datos 45

4. Lenguajes y manipulación de bases de datos 47

4.1. Lenguajes DDL y DML 48

4.2. Creación de bases de datos y tablas 50

4.3. Modificación de estructuras de tablas y columnas 52

4.4. Inserción, actualización y eliminación de registros 54

4.5. Eliminación de tablas y bases de datos 55

5. Gestión y administración de bases de datos 57

5.1. Sistemas gestores de bases de datos 57

5.2. Configuración básica de sistemas gestores de bases de datos 59

5.3. Herramientas de modelado de bases de datos 60

6. Procesamiento y almacenamiento de datos 62

6.1. Cargas de trabajo transaccionales 63

6.2. Cargas de trabajo analíticas 64

6.3. Principios ACID y gestión de transacciones 65

6.4. Tipos de almacenamiento de datos 66

7. Bases de datos NoSQL y sistemas distribuidos 68

7.1. Concepto y tipos de bases de datos NoSQL 69

7.2. Operaciones y manipulación de datos en bases de datos NoSQL 70

7.3. Consultas, filtrado y agregación de datos 72

7.4. Teorema CAP y sistemas distribuidos 73

Síntesis 76

Glosario 78

Referencias bibliográficas 80

Créditos 82

Introducción

El modelado y la gestión de bases de datos constituyen procesos fundamentales en el desarrollo de soluciones de software, ya que permiten organizar, almacenar y administrar la información de manera eficiente y segura. En este componente formativo se abordan los conceptos esenciales relacionados con las bases de datos, sus modelos, estructuras y tendencias actuales, integrando enfoques relacionales y NoSQL utilizados en diferentes contextos tecnológicos.

Asimismo, se estudian los principios del diseño conceptual, lógico y físico de bases de datos, la normalización y la interpretación de modelos relacionales, con el propósito de comprender la organización de la información y las relaciones entre entidades. De igual forma, se desarrollan conocimientos relacionados con los lenguajes de definición y manipulación de datos, las restricciones de integridad y las herramientas de gestión y modelado empleadas en entornos reales de desarrollo.

Finalmente, se analizan los diferentes tipos de almacenamiento de datos, las características de los sistemas distribuidos y los principios ACID y CAP, permitiendo reconocer las necesidades de procesamiento transaccional y analítico en soluciones modernas de software.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

Video 1. Modelado y gestión de bases de datos



Enlace de reproducción del video

Transcripción del video. Modelado y gestión de bases de datos
La historia de las bases de datos puede compararse con una gran biblioteca donde cada libro representa un dato y cada estante una forma de organizar la información. Durante muchos años, los registros se almacenaban en archivos físicos, lo que dificultaba encontrar información con rapidez y precisión. A medida que la tecnología avanzó y la cantidad de datos creció, surgió la necesidad de crear sistemas capaces de almacenar, organizar y consultar grandes volúmenes de información de manera eficiente. En este contexto aparecen las bases de datos, herramientas fundamentales que permiten estructurar la información para que pueda ser utilizada en diferentes entornos como empresas, plataformas digitales, sistemas académicos y aplicaciones tecnológicas.

Transcripción del video. Modelado y gestión de bases de datos
Para lograrlo, es necesario comprender cómo se organizan los datos, cómo se representan mediante modelos y cómo se transforman en estructuras que puedan implementarse en sistemas gestores de bases de datos. A partir de este proceso se crean tablas, se establecen relaciones entre los datos y se garantiza la integridad de la información. Posteriormente, se utilizan lenguajes especializados que permiten crear, modificar y manipular los datos almacenados. Gracias a estos lenguajes, los sistemas pueden registrar nueva información, actualizarla, consultarla o eliminarla según las necesidades de cada aplicación. En la actualidad, las bases de datos no solo almacenan información, sino que también permiten procesarla, analizarla y distribuirla en sistemas cada vez más complejos, incluyendo arquitecturas distribuidas y tecnologías NoSQL. Comprender estos fundamentos permite reconocer cómo los datos se transforman en información valiosa para el funcionamiento de los sistemas digitales y la toma de decisiones.

1. Fundamentos de bases de datos

Las bases de datos constituyen uno de los componentes esenciales en el desarrollo de soluciones de software, ya que permiten almacenar, organizar, consultar y administrar grandes volúmenes de información de manera eficiente. Su implementación facilita la gestión estructurada de los datos en diferentes contextos organizacionales, empresariales y tecnológicos, contribuyendo a la automatización de procesos y a la toma de decisiones basada en información confiable.

En la actualidad, las bases de datos evolucionan constantemente para responder a las necesidades de procesamiento, almacenamiento y análisis de información generadas por aplicaciones web, sistemas empresariales, dispositivos móviles, servicios en la nube y entornos distribuidos. Por esta razón, existen diferentes modelos y tecnologías orientadas a administrar datos estructurados, semiestructurados y desestructurados, adaptándose a diversos requerimientos de escalabilidad, disponibilidad y rendimiento.

1.1. Conceptos básicos y características de las bases de datos

Una base de datos es un conjunto organizado de datos relacionados entre sí, almacenados de forma estructurada para facilitar su acceso, administración, actualización y consulta. Su principal propósito consiste en gestionar información de manera eficiente, segura y confiable, permitiendo que diferentes usuarios y aplicaciones interactúen con los datos de forma controlada.

Las bases de datos son utilizadas en múltiples contextos, como sistemas empresariales, plataformas educativas, aplicaciones móviles, redes sociales, sistemas

bancarios, servicios de salud y comercio electrónico. Gracias a su capacidad de organización y procesamiento, permiten almacenar grandes cantidades de información y recuperar los datos de manera rápida y precisa.

Para comprender el funcionamiento de una base de datos, es necesario reconocer algunos conceptos fundamentales:

- **Dato**: representación básica de información, como números, textos, fechas o imágenes.
- **Información**: conjunto de datos procesados y organizados que adquieren significado.
- **Tabla**: estructura compuesta por filas y columnas utilizada para almacenar datos relacionados.
- **Registro**: fila de una tabla que contiene la información completa de un elemento específico.
- **Campo**: columna de una tabla que almacena un tipo particular de dato.
- **Entidad**: objeto o elemento del mundo real sobre el cual se almacena información.
- **Relación**: asociación existente entre dos o más entidades dentro de una base de datos.
- **Clave primaria**: atributo que identifica de manera única cada registro de una tabla.
- **Clave foránea**: atributo que establece relaciones entre tablas mediante referencias a claves primarias.

Las bases de datos poseen características que garantizan la organización y el manejo eficiente de la información:

- **Estructuración de datos**: organizan la información de manera lógica y ordenada.
- **Persistencia**: los datos permanecen almacenados incluso después de finalizar los procesos o apagar los sistemas.
- **Integridad**: aseguran que la información sea válida, coherente y consistente.
- **Seguridad**: permiten controlar el acceso a los datos mediante usuarios y permisos.
- **Disponibilidad**: facilitan el acceso oportuno a la información cuando es requerida.
- **Escalabilidad**: soportan el crecimiento de datos y usuarios sin afectar significativamente el rendimiento.
- **Redundancia controlada**: minimizan la duplicidad innecesaria de información.
- **Acceso concurrente**: permiten que varios usuarios interactúen simultáneamente con los datos.
- **Recuperación y respaldo**: ofrecen mecanismos para restaurar información ante fallos o pérdidas de datos.

Estas características convierten a las bases de datos en herramientas fundamentales para el almacenamiento y procesamiento de información en los sistemas de software actuales.

1.2. Evolución y tendencias de las bases de datos

La evolución de las bases de datos responde a la necesidad de almacenar, administrar y procesar volúmenes crecientes de información en diferentes contextos tecnológicos. A medida que las organizaciones incrementan el uso de sistemas digitales, las bases de datos evolucionan para ofrecer mayor capacidad de almacenamiento, velocidad de procesamiento, disponibilidad y escalabilidad.

Los primeros sistemas de almacenamiento de información se basan en archivos planos, donde los datos se guardan de manera independiente y con escasa relación entre sí. Este modelo presenta limitaciones relacionadas con la redundancia, la inconsistencia de la información y la dificultad para compartir datos entre aplicaciones.

Posteriormente surgen las bases de datos jerárquicas y de red, diseñadas para organizar la información mediante estructuras de relaciones predefinidas. Aunque mejoran la organización de los datos, presentan dificultades de flexibilidad y mantenimiento.

En la década de 1970, Edgar F. Codd propone el modelo relacional, el cual transforma la administración de datos mediante el uso de tablas relacionadas y el lenguaje SQL. Este enfoque facilita la consulta, manipulación y mantenimiento de la información, convirtiéndose en uno de los modelos más utilizados en sistemas empresariales y aplicaciones de software.

Con el crecimiento de Internet, las aplicaciones web y el manejo masivo de datos, aparecen nuevos modelos de almacenamiento como las bases de datos NoSQL, orientadas al procesamiento de datos semiestructurados y desestructurados. Estas tecnologías priorizan la escalabilidad, el rendimiento y la distribución de datos en múltiples servidores.

Más recientemente surgen las bases de datos NewSQL, que combinan características de escalabilidad propias de NoSQL con las propiedades transaccionales y de consistencia de las bases de datos relacionales tradicionales.

A. Tendencias actuales en bases de datos

Las tendencias modernas en bases de datos están orientadas a responder a las necesidades de procesamiento de grandes volúmenes de información y al funcionamiento de aplicaciones distribuidas y en tiempo real. Entre las principales tendencias se encuentran:

- **Bases de datos en la nube**: permiten almacenar y administrar información mediante servicios remotos accesibles desde Internet.
- **Procesamiento de Big Data**: facilita el análisis de grandes cantidades de datos generados por múltiples fuentes.
- **Bases de datos distribuidas**: almacenan información en diferentes servidores para mejorar la disponibilidad y tolerancia a fallos.
- **Bases de datos NoSQL**: optimizan el manejo de datos semiestructurados y desestructurados.

- **Automatización y administración inteligente**: incorporan herramientas de inteligencia artificial para optimizar tareas de mantenimiento y rendimiento.
- **Procesamiento en tiempo real**: permite analizar y consultar información de manera inmediata.
- **Seguridad y protección de datos**: fortalecen mecanismos de autenticación, cifrado y control de acceso.
- **Integración con analítica y ciencia de datos**: favorecen el procesamiento de información para la toma de decisiones.

La evolución de las bases de datos demuestra cómo las tecnologías de almacenamiento se adaptan continuamente a las necesidades de las organizaciones y de los entornos digitales modernos, permitiendo gestionar información de manera más eficiente, flexible y segura.

Referentes teóricos:

- Codd (1970) establece las bases del modelo relacional mediante la organización de los datos en tablas relacionadas, propuesta que transforma el desarrollo de los sistemas gestores de bases de datos modernos.
- Date (2004) señala que las bases de datos constituyen sistemas diseñados para almacenar información de forma estructurada, garantizando consistencia, integridad y facilidad de acceso.

- Silberschatz, Korth y Sudarshan (2020) destacan que las tendencias actuales en bases de datos están orientadas hacia la escalabilidad, el procesamiento distribuido y la administración eficiente de grandes volúmenes de información.

1.3. Modelos de datos estructurados, semiestructurados y desestructurados

Los modelos de datos representan la forma en que la información se organiza, almacena y procesa dentro de un sistema. Dependiendo de la estructura y el nivel de organización de los datos, estos pueden clasificarse en estructurados, semiestructurados y desestructurados. Cada modelo responde a diferentes necesidades de almacenamiento y procesamiento en entornos tecnológicos modernos.

a) **Datos estructurados**: son aquellos que se organizan siguiendo una estructura definida y uniforme, generalmente mediante tablas compuestas por filas y columnas. Este tipo de información utiliza esquemas previamente establecidos que facilitan su almacenamiento, consulta y manipulación.

Los datos estructurados son ampliamente utilizados en bases de datos relacionales, ya que permiten realizar búsquedas rápidas y garantizar la integridad de la información.

Características de los datos estructurados:

- Poseen una organización fija y definida.
- Utilizan tablas, registros y campos.

- Mantienen relaciones entre datos.
- Facilitan consultas mediante lenguaje SQL.
- Garantizan consistencia e integridad de la información.

La siguiente es una ejemplificación de estos datos:

Tabla 1. Ejemplo de datos estructurados

Id_cliente	Nombre	Ciudad	Edad
101	Laura Gómez.	Bogotá.	28.
102	Andrés Ruiz.	Medellín.	35.

En este ejemplo, la información se encuentra organizada en columnas específicas y cada registro mantiene la misma estructura.

b) Datos semiestructurados: presentan una organización parcial, es decir, no siguen un esquema rígido como las bases de datos relacionales, pero incorporan etiquetas o marcas que permiten identificar y organizar la información.

Este tipo de datos es común en aplicaciones web, servicios en la nube e intercambio de información entre sistemas.

Características de los datos semiestructurados:

- Poseen estructura flexible.
- Utilizan etiquetas o atributos descriptivos.
- Permiten diferentes tipos de datos en un mismo documento.

- Facilitan el intercambio de información entre aplicaciones.
- Son utilizados frecuentemente en bases de datos NoSQL.

Ejemplo en formato JSON:

```
{
  "cliente": "Laura Gómez",
  "ciudad": "Bogotá",
  "compras": [
    "Portátil",
    "Mouse",
    "Teclado"
  ]
}
```

En este caso, la información posee organización mediante etiquetas, pero no requiere una estructura fija en todos los registros.

c) **Datos desestructurados**: corresponden a información que no posee un formato definido ni una organización uniforme. Este tipo de datos representa gran parte de la información generada actualmente en Internet y en plataformas digitales.

Los sistemas modernos utilizan tecnologías especializadas para almacenar y procesar este tipo de información debido a su complejidad y volumen.

Características de los datos desestructurados:

- No siguen esquemas definidos.
- Presentan formatos variados.
- Requieren herramientas especializadas para su análisis.
- Generan grandes volúmenes de información.
- Son frecuentes en entornos de Big Data y analítica de datos.

Ejemplos de datos desestructurados:

- Imágenes.
- Videos.
- Audios.
- Publicaciones en redes sociales.
- Correos electrónicos.
- Documentos multimedia.
- Archivos PDF o Word sin estructura uniforme.

Por ejemplo, un video publicado en una plataforma digital contiene información visual, auditiva y textual que no puede organizarse fácilmente en tablas tradicionales.

Con el fin de identificar las principales diferencias entre los modelos de datos estructurados, semiestructurados y desestructurados, a continuación, se presenta una comparación relacionada con su organización, ejemplos y tecnologías utilizadas para su almacenamiento y procesamiento:

Tabla 2. Comparación de los modelos de datos

Tipo de dato	Estructura	Ejemplo	Tecnología frecuente
Estructurado.	Fija y organizada.	Tablas de clientes.	Bases de datos relacionales.
Semiestructurado.	Flexible con etiquetas.	JSON y XML.	Bases de datos NoSQL.
Desestructurado.	Sin estructura definida.	Videos e imágenes.	Big Data y almacenamiento distribuido.

La identificación de estos modelos de datos permite seleccionar las tecnologías y herramientas más adecuadas para almacenar, administrar y procesar información según las necesidades de cada sistema de software.

1.4. Bases de datos relacionales, NoSQL y NewSQL

Las bases de datos evolucionan constantemente para responder a las necesidades de almacenamiento, procesamiento y administración de información en diferentes entornos tecnológicos. Dependiendo de la estructura de los datos, el volumen de información y los requerimientos de escalabilidad y rendimiento, existen distintos modelos de bases de datos, entre los que se destacan las relacionales, NoSQL y NewSQL.

Cada uno de estos enfoques presenta características particulares que permiten su implementación en aplicaciones empresariales, sistemas distribuidos, plataformas web y soluciones orientadas al análisis de grandes volúmenes de datos.

a) Bases de datos relacionales

Organizan la información mediante tablas compuestas por filas y columnas, estableciendo relaciones entre los datos a través de claves primarias y claves foráneas. Este modelo se fundamenta en la teoría relacional propuesta por Edgar F. Codd y utiliza el lenguaje SQL para la consulta y manipulación de la información.

Estas bases de datos son ampliamente utilizadas en sistemas empresariales y aplicaciones que requieren consistencia, integridad y control de transacciones.

Características de las bases de datos relacionales:

- Utilizan tablas estructuradas.
- Implementan relaciones entre entidades.
- Emplean lenguaje SQL.
- Garantizan integridad y consistencia de los datos.
- Aplican principios ACID para la gestión de transacciones.
- Facilitan consultas complejas y estructuradas.

Ejemplos de bases de datos relacionales:

- MySQL.
- PostgreSQL.
- Oracle Database.
- Microsoft SQL Server.

La siguiente es una ejemplificación de estos datos:

Tabla 3. Ejemplo de estructura relacional

Id_cliente	Nombre	Ciudad
101	Laura Gómez.	Bogotá.
102	Andrés Ruiz.	Medellín.

En este ejemplo, los datos se organizan de manera estructurada en tablas con atributos definidos.

b) Bases de datos NoSQL

Surgen como alternativa a los sistemas relacionales tradicionales, especialmente para manejar grandes volúmenes de datos distribuidos, semiestructurados y desestructurados. Estas bases de datos priorizan la escalabilidad, disponibilidad y flexibilidad en el almacenamiento de información.

El término NoSQL significa “Not Only SQL”, indicando que estos sistemas pueden utilizar diferentes modelos de almacenamiento distintos al modelo tabular tradicional.

Características de las bases de datos NoSQL:

- Manejan datos estructurados, semiestructurados y desestructurados.
- Permiten escalabilidad horizontal.
- Ofrecen alta disponibilidad y rendimiento.
- Utilizan esquemas flexibles.
- Son utilizadas en aplicaciones web, redes sociales y big data.
- Tipos de bases de datos NoSQL.
- Bases de datos orientadas a documentos.

- Bases de datos clave-valor.
- Bases de datos orientadas a columnas.
- Bases de datos orientadas a grafos.

Ejemplos de bases de datos NoSQL:

- MongoDB.
- Cassandra.
- Redis.
- Neo4j.

Ejemplo de documento NoSQL en formato JSON:

```
{
  "cliente": "Laura Gómez",
  "ciudad": "Bogotá",
  "productos": [
    "Portátil",
    "Teclado"
  ]
}
```

En este modelo, la información se almacena de forma flexible sin requerir una estructura fija para todos los registros.

Para complementar la importancia de NoSQL, se recomienda acceder al siguiente video: https://www.youtube.com/watch?v=M-Iw1awJ1NU

c) Bases de datos NewSQL

Aparecen como una solución que combina características de las bases de datos relacionales tradicionales con capacidades de escalabilidad propias de los sistemas NoSQL. Su objetivo consiste en mantener las propiedades ACID y el uso de SQL, mientras soportan grandes volúmenes de datos y arquitecturas distribuidas.

Estas tecnologías son utilizadas en aplicaciones modernas que requieren alto rendimiento transaccional y procesamiento distribuido.

Características de las bases de datos NewSQL:

- Utilizan lenguaje SQL.
- Mantienen propiedades ACID.
- Permiten escalabilidad horizontal.
- Soportan arquitecturas distribuidas.
- Optimizan el procesamiento de transacciones en tiempo real.

Ejemplos de bases de datos NewSQL:

- Google Spanner.
- CockroachDB.
- VoltDB.
- NuoDB.

Con el propósito de identificar las principales diferencias entre las bases de datos relacionales, NoSQL y NewSQL, a continuación, se presenta una comparación relacionada con su estructura, escalabilidad, manejo de datos y aplicaciones más frecuentes:

Tabla 4. Comparación entre bases de datos relacionales, NoSQL y NewSQL

Característica	Relacionales	NoSQL	NewSQL
Estructura de datos.	Tablas.	Flexible.	Tablas.
Lenguaje principal.	SQL.	Variable.	SQL.
Escalabilidad.	Vertical.	Horizontal.	Horizontal.
Integridad ACID.	Sí.	Parcial o limitada.	Sí.
Tipo de datos.	Estructurados.	Semi y desestructurados.	Estructurados.
Uso frecuente.	Sistemas empresariales.	Big Data y aplicaciones web.	Sistemas distribuidos modernos.

La selección de un modelo de base de datos depende de las necesidades del sistema, el tipo de información administrada y los requerimientos de disponibilidad, escalabilidad y rendimiento de las aplicaciones de software.

2. Modelado de bases de datos

El modelado de bases de datos es un proceso fundamental en el desarrollo de sistemas de información, ya que permite representar de manera estructurada la forma en que los datos se organizan, relacionan y almacenan dentro de un sistema. Este proceso facilita la comprensión de la información que debe gestionarse y contribuye a diseñar bases de datos eficientes, coherentes y adaptadas a los requerimientos del software.

A través del modelado de datos se identifican las entidades, atributos y relaciones que forman parte de un sistema, permitiendo establecer una representación conceptual de la información antes de su implementación en un sistema gestor de bases de datos. De esta manera, se reduce la redundancia de datos, se mejora la integridad de la información y se optimiza el acceso a los datos.

El proceso de modelado generalmente se desarrolla en tres niveles principales:

- Diseño conceptual.
- Diseño lógico.
- Diseño físico.

Cada uno de estos niveles permite describir la estructura de la base de datos desde diferentes perspectivas, comenzando por una representación general del sistema hasta su implementación técnica en un gestor de bases de datos.

En este apartado se abordan los principales elementos y etapas del modelado de bases de datos, incluyendo el diseño conceptual, el modelo entidad-relación, la transformación hacia el modelo lógico, el diseño físico y el proceso de normalización,

aspectos fundamentales para la construcción de bases de datos eficientes y bien estructuradas.

2.1. Diseño conceptual de bases de datos

El diseño conceptual de bases de datos corresponde a la primera etapa del proceso de modelado de datos. En esta fase se identifican y describen los elementos principales de la información que el sistema debe gestionar, sin considerar aún aspectos técnicos relacionados con el sistema gestor de bases de datos o la implementación tecnológica.

El objetivo del diseño conceptual consiste en representar de manera clara y comprensible la estructura de la información, identificando las entidades, los atributos y las relaciones que existen entre los diferentes elementos del sistema. Esta representación permite comprender cómo se organiza la información y cómo interactúan los distintos componentes dentro del contexto de una aplicación o sistema de información.

Durante esta etapa se analiza el problema o necesidad del sistema, se identifican los datos relevantes y se establece una visión general del modelo de información. De esta manera, el diseño conceptual facilita la comunicación entre analistas, desarrolladores y expertos del dominio, permitiendo validar la estructura de los datos antes de avanzar hacia fases más técnicas del diseño.

Entre los principales elementos que se identifican en el diseño conceptual se encuentran:

- **Entidades**: representa objetivos o conceptos del mundo real sobre las cuales se desea almacenar información, como por ejemplo cliente, productos, pedidos o empleados.
- **Atributos**: son las características o prioridades que describen a una entidad. Por ejemplo, una entidad cliente puede tener atributos como nombre, identificación, correo electrónico o ciudad.
- **Relaciones**: indican la forma en que las entidades se vinculan entre si dentro del sistema. Por ejemplo, un cliente puede realizar uno o varios pedidos, lo que establece una relación entre las entidades cliente y pedido.

Ejemplo de diseño conceptual:

En un sistema de gestión de ventas se pueden identificar las siguientes entidades:

- Cliente.
- Pedido.
- Producto.

Las relaciones entre estas entidades pueden representarse de la siguiente manera:

- Un cliente realiza pedidos.
- Un pedido puede incluir varios productos.

Este tipo de representación permite visualizar de forma general cómo se organiza la información dentro del sistema antes de convertir el modelo en tablas o estructuras físicas.

2.2. Modelo Entidad-Relación (ER)

El modelo Entidad-Relación (ER) es una técnica ampliamente utilizada para representar de forma gráfica la estructura de una base de datos durante la fase de diseño conceptual. Este modelo permite describir los elementos principales del sistema de información y las relaciones existentes entre ellos, facilitando la comprensión de cómo se organizan y conectan los datos.

El modelo ER fue propuesto por Peter Chen en 1976, y desde entonces se ha convertido en uno de los métodos más utilizados para el diseño de bases de datos, debido a su capacidad para representar de manera clara y estructurada los componentes de un sistema de información.

En este modelo se utilizan diagramas que encuentran las entidades, los atributos y las relaciones, permitiendo representar la estructura de los datos antes de su implementación en un sistema gestor de bases de datos.

Elementos del modelo Entidad-Relación:

a) **Entidad**: representa un objeto, concepto o elemento del mundo real sobre el cual se desea almacenar información. Las entidades suelen representarse mediante rectángulos en los diagramas ER.

Ejemplo de entidades:

- Cliente.
- Producto.
- Pedido.

b) **Atributos**: corresponden a las propiedades o características que describen una entidad. En los diagramas ER se representan mediante óvalos conectados a la entidad correspondiente.

Ejemplo de atributos para la entidad cliente:

- Identificación.
- Nombre.
- Dirección.
- Teléfono.

c) **Relaciones**: indican la forma en que las entidades se vinculan entre sí dentro de un sistema. En los diagramas ER se representan mediante rombos o conectores entre entidades.

Ejemplo de relación:

- Un cliente realiza pedidos.

Cardinalidad de las relaciones

Indica la cantidad de instancias de una entidad que pueden relacionarse con otra entidad. Entre las cardinalidades más comunes se encuentran:

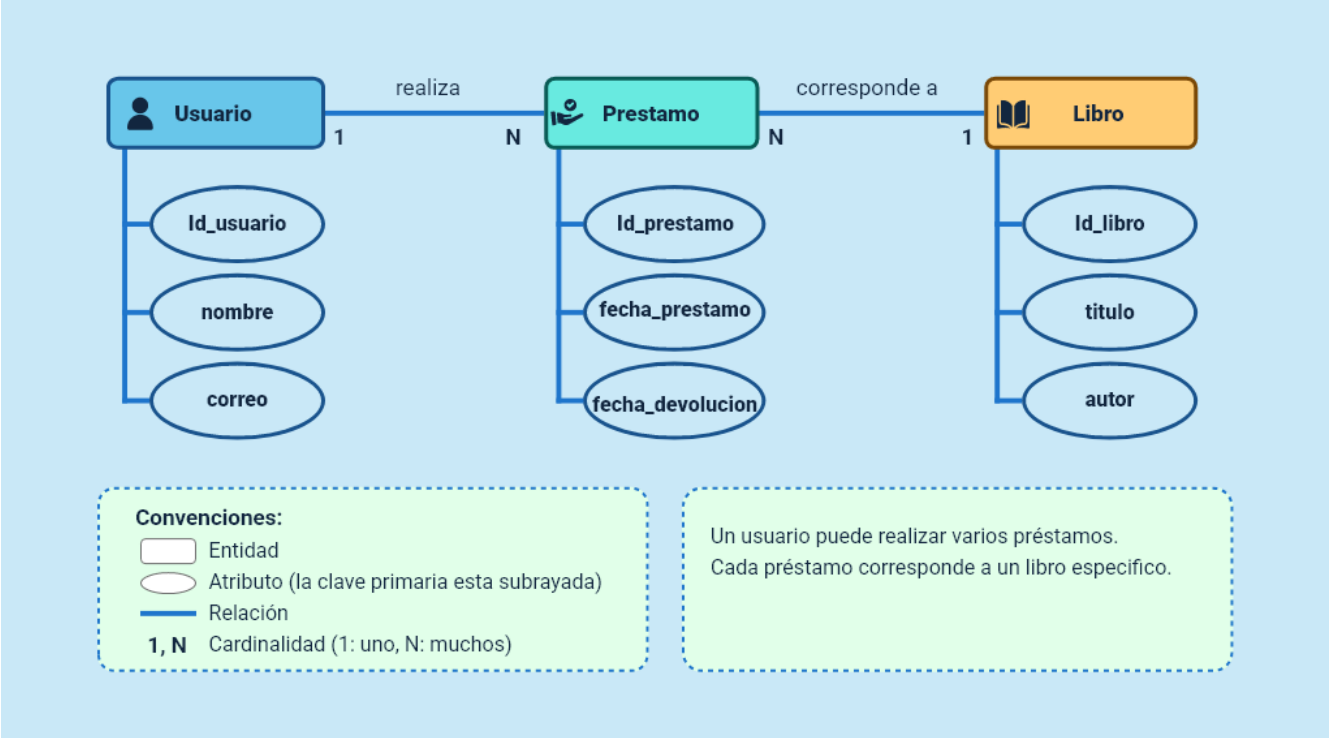
- **Uno a uno (1:1)**: un registro de una entidad se relaciona con un solo registro de otra entidad.
- **Uno a muchos (1:N)**: un registro de una entidad puede relacionarse con varios registros de otra entidad.
- **Muchos a muchos (N:M)**: varios registros de una entidad pueden relacionarse con varios registros de otra entidad.

Para comprender cómo funciona el modelo Entidad-Relación, se presenta un ejemplo sencillo basado en un sistema de préstamos de biblioteca. Este tipo de representación permite identificar las entidades principales del sistema, sus atributos y las relaciones que existen entre ellas, facilitando la organización y estructuración de la información en una base de datos.

En este ejemplo se identifican tres entidades principales: Usuario, Libro y Préstamo. Cada una contiene atributos que describen sus características. Además, las entidades se relacionan entre sí para representar cómo interactúan los datos dentro del sistema.

Por ejemplo, un usuario realiza un préstamo y cada préstamo corresponde a un libro determinado. De esta manera, el modelo permite visualizar cómo se organizan los datos y cómo se conectan entre sí dentro del sistema.

Figura 1. Modelo Entidad-Relación de un sistema de préstamos de biblioteca



Esto indica que un usuario puede realizar varios préstamos y cada préstamo corresponde a un libro específico.

Elementos del modelo Entidad-Relación:

- **Entidad**: una entidad es cualquier objeto, persona o concepto del mundo real sobre el cual se desea almacenar información dentro de una base de datos. En el ejemplo presentado, Usuario, Libro y Préstamo representan entidades del sistema.
- **Atributo**: un atributo es una característica que describe a una entidad. Los atributos permiten registrar información específica sobre cada elemento.

Por ejemplo, la entidad Usuario puede tener atributos como nombre y correo.

- **Relación**: una relación es la asociación que existe entre dos o más entidades y permite representar cómo interactúan dentro del sistema. En el ejemplo, la relación realiza conecta las entidades Usuario y Préstamo.

Para complementar la información de Entidad-Relación, se recomienda acceder al siguiente video: https://www.youtube.com/watch?v=t7KZFiCstwI

2.3. Diseño lógico de bases de datos

El diseño lógico de bases de datos corresponde a la etapa en la que se define la estructura lógica que tendrá la información dentro de un sistema gestor de bases de datos. En esta fase se organizan los datos en tablas, columnas y relaciones, siguiendo las reglas del modelo de datos que se utilizará, generalmente el modelo relacional.

El objetivo del diseño lógico es representar los datos de manera estructurada y coherente, permitiendo establecer las entidades, los atributos y las relaciones que formarán parte de la base de datos. En este nivel se definen elementos importantes como las claves primarias (PK, Primary Key), las claves foráneas (FK, Foreign Key) y las restricciones que garantizan la integridad de la información.

A diferencia del diseño físico, el diseño lógico no se centra en aspectos técnicos relacionados con el almacenamiento o el rendimiento del sistema, sino en la organización estructural de los datos. Esto permite construir un modelo claro y consistente que servirá como base para la implementación posterior de la base de datos.

Para comprender mejor este proceso, a continuación, se presenta un ejemplo sencillo de cómo una entidad del modelo Entidad-Relación puede representarse en el diseño lógico mediante una tabla:

Tabla 5. Transformación de una entidad del modelo ER a una tabla

Paso	Descripción	Representación
Paso 1. Identificar la entidad.	Se identifica la entidad y sus atributos en el modelo conceptual.	Entidad: Aprendiz → atributos: id_aprendiz, nombre, documento, correo.
Paso 2. Convertir la entidad en tabla.	La entidad se transforma en una tabla dentro del modelo lógico.	Tabla: Aprendiz.
Paso 3. Definir columnas.	Cada atributo de la entidad se convierte en una columna de la tabla.	id_aprendiz, nombre, documento, correo.
Paso 4. Establecer la clave primaria.	Se define el atributo que identificará de forma única cada registro.	id_aprendiz (PK).

Como resultado, la entidad conceptual Aprendiz se convierte en una tabla de la base de datos, donde cada fila representa un registro y cada columna corresponde a un atributo de la entidad. Este proceso permite estructurar la información de manera organizada para su posterior implementación en el sistema gestor de bases de datos.

2.4. Transformación del modelo conceptual al modelo lógico

La transformación del modelo conceptual al modelo lógico es el proceso mediante el cual la información identificada durante el análisis del sistema se organiza en una estructura que pueda ser implementada dentro de una base de datos.

En el modelo conceptual se describen los elementos principales del sistema, como las entidades, sus atributos y las relaciones que existen entre ellas. Sin embargo, esta representación todavía no define cómo se almacenarán los datos dentro de un sistema gestor de bases de datos.

Durante la transformación al modelo lógico, estos elementos se convierten en tablas, columnas y claves que permiten estructurar la información de manera organizada. De esta forma, las entidades del modelo conceptual se transforman en tablas, los atributos en columnas y las relaciones se representan mediante claves primarias y claves foráneas.

Este proceso permite pasar de una representación conceptual del sistema a una estructura lógica preparada para su implementación en una base de datos.

Ejemplo de transformación: supóngase un sistema sencillo de biblioteca donde se identifican las siguientes entidades en el modelo conceptual:

- Usuario.
- Libro.
- Préstamo.

Cada entidad posee atributos que describen sus características. Por ejemplo:

- **Usuario**: id_usuario, nombre, correo.

- **Libro**: id_libro, titulo, autor.
- **Préstamo**: id_prestamo, fecha_prestamo.

Durante la transformación al modelo lógico, estas entidades se convierten en tablas dentro de la base de datos:

Tabla 6. Ejemplo de transformación del modelo conceptual al modelo lógico

Tabla	Columnas	Clave primaria
Usuario.	id_usuario, nombre, correo.	id_usuario (PK).
Libro.	id_libro, titulo, autor.	id_libro (PK).
Prestamo.	id_prestamo, fecha_prestamo, id_usuario, id_libro.	id_prestamo (PK).

En este caso, la tabla Préstamo incluye los campos id_usuario y id_libro, los cuales funcionan como claves foráneas (FK) que permiten relacionar los préstamos con el usuario que realiza el préstamo y con el libro prestado.

Este proceso garantiza que la información del sistema quede organizada de forma estructurada, permitiendo gestionar los datos de manera eficiente dentro de una base de datos.

2.5. Diseño físico de bases de datos

El diseño físico de bases de datos corresponde a la etapa en la que se define cómo se almacenarán y gestionarán los datos dentro del sistema gestor de bases de datos. En esta fase se pasa del modelo lógico a la implementación técnica, teniendo en

cuenta aspectos como el almacenamiento, el rendimiento y la eficiencia en el acceso a la información.

Mientras que el diseño lógico se enfoca en la estructura de las tablas, atributos y relaciones, el diseño físico se centra en cómo esas estructuras se implementan realmente dentro del sistema, considerando las características del gestor de bases de datos utilizado, el volumen de información y la forma en que los usuarios accederán a los datos.

En esta etapa se definen elementos importantes como:

- **Tipos de datos**: especifican el formato en que se almacenara cada dato dentro de una tabla, por ejemplo, números, texto o fechas.
- **Índices**: estructura que permiten acelerar las consultas y mejorar el rendimiento de la base de datos.
- **Restricciones de integridad**: reglas que garantizan que los datos almacenados sean válidos y coherente.
- **Estructura de almacenamiento**: organización física de los archivos o bloques donde se guardará la información.

El diseño físico también busca optimizar el desempeño del sistema, permitiendo que las consultas se ejecuten de forma rápida y eficiente, especialmente cuando la base de datos maneja grandes volúmenes de información.

A continuación, se presenta un ejemplo sencillo de cómo se define la estructura física de una tabla dentro de una base de datos:

Tabla 7. Ejemplo de diseño físico de una tabla

Campo	Tipo de dato	Descripción
id_usuario	INT	Identificador único del usuario.
Nombre	VARCHAR(100)	Nombre del usuario.
Correo	VARCHAR(100)	Correo electrónico del usuario.
fecha_registro	DATE	Fecha en que el usuario se registró.

En este ejemplo, además de definir las columnas de la tabla, se especifica el tipo de dato que tendrá cada campo, lo que permite al sistema gestor de bases de datos almacenar y procesar la información de manera adecuada.

De esta forma, el diseño físico convierte la estructura lógica de la base de datos en una implementación real dentro del sistema, garantizando un almacenamiento eficiente, seguro y organizado de la información.

2.6. Normalización de bases de datos

La normalización de bases de datos es un proceso que permite organizar la información de manera eficiente, con el fin de evitar redundancias, reducir inconsistencias y garantizar la integridad de los datos. Este proceso consiste en estructurar las tablas y las relaciones entre ellas siguiendo una serie de reglas conocidas como formas normales.

Cuando una base de datos no está normalizada, es común encontrar datos repetidos o inconsistentes, lo que puede generar errores al momento de insertar, actualizar o eliminar información. Por esta razón, la normalización busca dividir la información en varias tablas relacionadas, de modo que cada dato se almacene una sola vez.

Entre los principales beneficios de la normalización se encuentran:

- Reduce la duplicación de datos.
- Mejora la organización de la información.
- Facilita el mantenimiento y actualización de los datos.
- Garantiza mayor integridad y consistencia en la base de datos.

Las formas normales más utilizadas en el diseño de bases de datos son las siguientes:

- **Primera forma normal (1FN)**

Se cumple cuando cada campo de la tabla contiene valores atómicos, es decir, un solo valor por celda, y no existen grupos repetidos de datos.

- **Segunda forma normal (2FN**)

Se alcanza cuando la tabla ya cumple con la primera forma normal, y demás, todos los atributos dependen completamente de la clave primaria, evitando dependencias parciales.

- **Tercera forma normal (3FN)**

Se cumple la tabla cuando está en segunda forma normal y los atributos que no dependen de otros atributos que no sean la clave primaria, eliminando dependencias transitivas.

Para comprender mejor este proceso, se presenta un ejemplo sencillo:

Tabla 8. Ejemplo de normalización de datos

Tabla no normalizada	Problema identificado
id_usuario, nombre_usuario, libro, autor_libro.	Un mismo usuario puede tener varios libros, lo que genera repetición de datos.

Después de aplicar la normalización, la información puede organizarse de la siguiente manera, mediante las tablas Usuario, Libro y Préstamo:

Tabla 9. Usuario

Tabla Usuario	Columnas
Usuario.	id_usuario (PK), nombre_usuario.

Tabla 10. Libro

Tabla Libro	Columnas
Libro.	id_libro (PK), titulo, autor.

Tabla 11. Préstamo

Tabla Prestamo	Columnas
Prestamo.	id_prestamo (PK), id_usuario, id_libro.

En este caso, la información se divide en varias tablas relacionadas, evitando la duplicación de datos y facilitando la gestión de la base de datos.

En conclusión, la normalización es una técnica fundamental en el diseño de bases de datos, ya que permite estructurar la información de forma lógica y organizada, contribuyendo a la creación de sistemas más eficientes, consistentes y fáciles de mantener.

3. Interpretación y estructura del modelo relacional

El modelo relacional es uno de los enfoques más utilizados para organizar y gestionar la información en las bases de datos. Este modelo propone representar los datos mediante tablas, también llamadas relaciones, las cuales se componen de filas y columnas. Cada tabla almacena información sobre una entidad específica y permite establecer vínculos con otras tablas mediante claves.

En este modelo, cada fila representa un registro o instancia de datos, mientras que cada columna corresponde a un atributo que describe una característica de la entidad. La organización estructurada de la información facilita su almacenamiento, consulta y manipulación mediante lenguajes de consulta como SQL.

El modelo relacional también establece reglas que garantizan la integridad y consistencia de los datos, evitando duplicidades o relaciones incorrectas entre las tablas. Para ello se utilizan mecanismos como las claves primarias, las claves foráneas y diferentes tipos de restricciones que controlan la validez de la información almacenada.

Comprender la estructura del modelo relacional permite interpretar correctamente cómo se organizan los datos dentro de una base de datos y cómo se relacionan entre sí las diferentes tablas. Este conocimiento resulta fundamental para el diseño, implementación y administración eficiente de sistemas de información basados en bases de datos relacionales.

3.1. Entidades, atributos y relaciones

En el modelo relacional, la información se organiza en estructuras que permiten representar y gestionar los datos dentro de una base de datos. Para comprender esta

estructura es importante interpretar cómo los elementos definidos en el modelo conceptual se reflejan en el modelo relacional.

Las entidades representan objetos o elementos del mundo real sobre los cuales se desea almacenar información. En el modelo relacional, cada entidad se transforma en una tabla dentro de la base de datos. Por ejemplo, en un sistema de biblioteca pueden existir entidades como Usuario, Libro o Préstamo, que posteriormente se representan como tablas que almacenan los registros correspondientes.

Los atributos corresponden a las características o propiedades que describen a una entidad. Dentro del modelo relacional, los atributos se representan como columnas de una tabla. Cada columna almacena un tipo específico de información, como el nombre de un usuario, el título de un libro o la fecha de un préstamo.

Las relaciones describen la forma en que las entidades se vinculan entre sí. En el modelo relacional, estas relaciones se implementan mediante claves que permiten conectar las tablas, garantizando que la información mantenga coherencia entre los distintos elementos del sistema.

Comprender la relación entre entidades, atributos y relaciones dentro del modelo relacional permite estructurar adecuadamente las tablas de la base de datos y asegurar que los datos se encuentren organizados de forma clara, consistente y fácil de consultar.

3.2. Llaves primarias y foráneas

En el modelo relacional, las tablas utilizan diferentes tipos de llaves para identificar los registros y establecer relaciones entre los datos almacenados. Las más

importantes son la llave primaria y la llave foránea, ya que permiten organizar la información y mantener la coherencia entre las tablas de una base de datos.

- **La llave primaria (Primary Key - PK)**

Es el atributo o conjunto de atributos que identifica de forma única cada registro dentro de una tabla. Sus valores no pueden repetirse ni ser nulos. Por ejemplo, en una tabla llamada Usuario, el campo id_usuario puede funcionar como llave primaria. permitiendo identificar de manera única a cada usuario registrado.

- **Llave foránea (Foreign Key - FK)**

Es un atributo que establece una relación entre dos tablas. Este campo hace referencia a la llave primaria de otra tabla, permitiendo conectar la información almacenada en diferentes entidades. Por ejemplo, en una tabla llamado Prestamo, el campo id_usuario puede funcionar como llave foránea que hace referencia al id_usuario de la tabla Usuario.

El uso de estas llaves permite mantener la relación entre las tablas y facilita la consulta, actualización y organización de la información dentro de una base de datos relacional. Para comprender mejor cómo funcionan las llaves primarias y foráneas dentro de un modelo relacional, a continuación, se presenta un ejemplo sencillo:

Tabla 12. Ejemplo de llaves primarias y foráneas en un modelo relacional

Tabla	Columnas	Llave primaria	Llave foránea
Usuario.	id_usuario, nombre, correo	id_usuario (PK).	—
Libro.	id_libro, titulo, autor	id_libro (PK).	—

Tabla	Columnas	Llave primaria	Llave foránea
Prestamo.	id_prestamo, fecha_prestamo, id_usuario, id_libro	id_prestamo (PK).	id_usuario (FK), id_libro (FK).

En este ejemplo, cada tabla posee una llave primaria (PK) que permite identificar de manera única cada registro. La tabla Préstamo incluye además llaves foráneas (FK) que hacen referencia a las llaves primarias de las tablas Usuario y Libro, permitiendo establecer la relación entre los datos.

De esta forma, el sistema puede registrar qué usuario realizó un préstamo y qué libro fue prestado, manteniendo la información organizada y relacionada dentro de la base de datos.

3.3. Integridad de identidad e integridad referencial

En una base de datos relacional, la integridad de los datos es un principio fundamental que garantiza que la información almacenada sea válida, coherente y confiable. Para lograrlo, los sistemas gestores de bases de datos utilizan diferentes reglas que permiten controlar la forma en que se registran y relacionan los datos. Entre las más importantes se encuentran la integridad de identidad y la integridad referencial.

- **Integridad de identidad**

Establece que cada registro dentro de una tabla debe ser identificado de manera única mediante una llave primaria (Primary Key o PK). Esto significa que el valor de la

llave primaria no puede repetirse ni quedar vacío, ya que su función es distinguir cada registro de los demás.

Por ejemplo, en una tabla llamada Usuario, el atributo id_usuario puede utilizarse como llave primaria. De esta manera, cada usuario tendrá un identificador único dentro de la base de datos, lo que permite evitar duplicidad de registros y facilita la organización de la información.

- **Integridad referencial**

Garantiza la coherencia de las relaciones entre las tablas de una base de datos. Este principio se cumple mediante el uso de llaves foráneas (Foreign Key o FK), las cuales permiten conectar registros de diferentes tablas.

Una llave foránea es un atributo que hace referencia a la llave primaria de otra tabla. Su función es asegurar que los valores registrados correspondan a datos que realmente existen en la tabla relacionada.

Por ejemplo, en una tabla Prestamo pueden existir los atributos id_usuario e id_libro como llaves foráneas. Estos valores deben corresponder a usuarios y libros previamente registrados en las tablas Usuario y Libro. De esta forma se evita registrar préstamos asociados a usuarios o libros inexistentes.

El cumplimiento de estas reglas permite mantener la consistencia de los datos, evitar errores en las relaciones entre tablas y garantizar que la información almacenada en la base de datos sea confiable y correctamente estructurada.

3.4. Restricciones y consistencia de los datos

En las bases de datos relacionales, las restricciones son reglas que se establecen para controlar los valores que pueden almacenarse en las tablas. Estas reglas permiten garantizar que la información registrada cumpla determinadas condiciones y que los datos mantengan coherencia dentro del sistema.

Las restricciones contribuyen a mantener la consistencia de los datos, es decir, aseguran que la información almacenada sea válida, completa y acorde con las reglas definidas para la base de datos. De esta manera, se evitan errores como registros incompletos, duplicados o valores que no corresponden con la estructura del sistema.

Entre las restricciones más comunes se encuentran las siguientes:

- **NOT NULL**: establece que un atributo no puede quedar vacío, por lo que siempre debe contener un valor.
- **UNIQUE**: garantiza que los valores de una columna no se repitan dentro de la tabla.
- **PRIMARY KEY (PK)**: identifica de manera única cada registro de una tabla y no permite valores nulos ni duplicados.
- **FOREIGN KEY (FK)**: asegura la relación entre tablas al exigir que los valores correspondan a registros existentes en otra tabla.
- **CHECK**: permite definir una condición que los valores deben cumplir para poder almacenarse en la base de datos.

La aplicación de estas restricciones fortalece la calidad de la información y facilita la administración de los datos dentro de los sistemas de información. Además, permite

mantener la coherencia entre las diferentes tablas y garantizar que los registros cumplan con las reglas definidas en el diseño de la base de datos.

4. Lenguajes y manipulación de bases de datos

La manipulación de bases de datos se realiza mediante lenguajes especializados que permiten definir, administrar y consultar la información almacenada en los sistemas gestores de bases de datos. Estos lenguajes forman parte del Structured Query Language (SQL), ampliamente utilizado para interactuar con bases de datos relacionales.

A través de estos lenguajes es posible crear la estructura de la base de datos, modificar tablas, insertar información, consultar registros y eliminar datos cuando sea necesario. Estas acciones permiten administrar de manera organizada la información y garantizar su correcta gestión dentro de los sistemas de información.

Los lenguajes utilizados para trabajar con bases de datos se agrupan generalmente en dos categorías principales. Por un lado, se encuentran los lenguajes orientados a la definición de la estructura de los datos, que permiten crear y modificar la organización de la base de datos. Por otro lado, existen los lenguajes destinados a la manipulación de los datos, que permiten insertar, actualizar, consultar o eliminar la información almacenada.

Comprender el funcionamiento de estos lenguajes es fundamental para gestionar adecuadamente una base de datos, ya que permiten controlar tanto la estructura como el contenido de la información. En los siguientes apartados se describen los principales tipos de lenguajes utilizados en la administración de bases de datos y las operaciones más comunes que se realizan mediante ellos.

4.1. Lenguajes DDL y DML

El Structured Query Language (SQL) es el lenguaje utilizado para crear, administrar y manipular bases de datos relacionales. Dentro de SQL existen diferentes tipos de instrucciones que permiten gestionar tanto la estructura como la información almacenada en la base de datos.

Entre los más importantes se encuentran el Data Definition Language (DDL) y el Data Manipulation Language (DML).

a) Data Definition Language (DDL)

Es el conjunto de comandos que permite definir y administrar la estructura de la base de datos. A través de este lenguaje se crean y modifican los objetos que almacenan la información, como tablas, columnas y restricciones.

El DDL es fundamental en el proceso de diseño de bases de datos, ya que permite implementar la estructura definida en el modelo lógico dentro del sistema gestor de bases de datos.

Entre sus funciones principales se encuentran:

- Crear estructuras de datos como bases de datos y tablas.
- Modificar estructuras existentes.
- Eliminar objetos de la base de datos.
- Definir restricciones que garantizan la integridad de los datos.

Los comandos más utilizados del DDL son:

Tabla 13. Comandos del DDL

Comando	Función
CREATE.	Permite crear bases de datos, tablas u otros objetos.
ALTER.	Permite modificar la estructura de una tabla existente.
DROP.	Permite eliminar objetos de la base de datos.
TRUNCATE.	Elimina todos los registros de una tabla sin borrar su estructura.

Ejemplo de creación de una tabla:

```
CREATE TABLE Estudiante (
   id_estudiante INT PRIMARY KEY,
   nombre VARCHAR(50),
   edad INT
);
```

b) Data Manipulation Language (DML)

Corresponde al conjunto de comandos que permiten gestionar la información almacenada en las tablas de una base de datos.

A través del DML es posible insertar, consultar, modificar o eliminar registros dentro de las tablas.

Las operaciones principales del DML son:

- Insertar nuevos registros.

- Consultar información almacenada.
- Actualizar datos existentes.
- Eliminar registros cuando sea necesario.

Los comandos más utilizados del DML son:

Tabla 14. Comandos del DML

Comando	Función
INSERT.	Permite agregar nuevos registros a una tabla.
SELECT.	Permite consultar información almacenada.
UPDATE.	Permite modificar datos existentes.
DELETE.	Permite eliminar registros de una tabla.

Ejemplo de inserción de datos:

```
INSERT INTO Estudiante (id_estudiante, nombre, edad)
VALUES (1, 'Ana', 20);
```

En conjunto, los lenguajes DDL y DML permiten administrar tanto la estructura de la base de datos como la información que se almacena en ella, siendo elementos fundamentales para el desarrollo y funcionamiento de los sistemas de información.

4.2. Creación de bases de datos y tablas

La creación de bases de datos y tablas constituye uno de los primeros pasos en la implementación de un sistema de información. En esta etapa se definen las estructuras

que permitirán almacenar, organizar y gestionar los datos dentro del sistema gestor de bases de datos.

Una base de datos es un conjunto organizado de información que se almacena de manera estructurada para facilitar su consulta, administración y actualización. Dentro de una base de datos se crean tablas, las cuales permiten guardar los datos relacionados con una entidad específica del sistema.

Cada tabla está compuesta por columnas y filas. Las columnas representan los atributos o características de la información que se desea registrar, mientras que las filas corresponden a cada uno de los registros almacenados.

Para realizar estas operaciones se utilizan comandos del Data Definition Language (DDL) dentro del lenguaje Structured Query Language (SQL).

a) **Creación de una base de datos**: el comando CREATE DATABASE permite crear una nueva base de datos en el sistema gestor.

Por ejemplo:

CREATE DATABASE Biblioteca;

Este comando crea una base de datos llamada Biblioteca, que servirá como contenedor para las tablas y demás objetos relacionados con el sistema.

b) **Creación de una tabla**: una vez creada la base de datos, es posible crear tablas que permitan almacenar la información. Para ello se utiliza el comando CREATE TABLE.

Por ejemplo:

CREATE TABLE Usuario (

```
    id_usuario INT PRIMARY KEY,
    nombre VARCHAR(100),
    correo VARCHAR(100)
);
```

En este ejemplo se crea la tabla Usuario, la cual contiene tres columnas:

- id_usuario: identifica de manera única a cada usuario y funciona como clave primaria.
- nombre: almacena el nombre del usuario.
- correo: registra la dirección de correo electrónico.

La creación adecuada de bases de datos y tablas permite establecer una estructura organizada para el almacenamiento de la información, facilitando posteriormente la manipulación, consulta y administración de los datos dentro del sistema.

4.3. Modificación de estructuras de tablas y columnas

En el proceso de administración de una base de datos, es común que la estructura de las tablas necesite ajustarse con el tiempo. Estas modificaciones pueden surgir por nuevos requerimientos del sistema, cambios en los procesos de información o la necesidad de agregar nuevos atributos a los datos almacenados.

La modificación de la estructura de las tablas se realiza mediante el comando ALTER TABLE, el cual forma parte del Data Definition Language (DDL) dentro del

Structured Query Language (SQL). Este comando permite agregar, modificar o eliminar columnas en una tabla existente sin afectar la base de datos completa.

a) **Agregar una nueva columna**: el comando ADD permite incorporar un nuevo atributo a una tabla. Por ejemplo:

```
ALTER TABLE Usuario

ADD telefono VARCHAR(20);
```

En este caso se agrega la columna telefono a la tabla Usuario, permitiendo registrar un número de contacto para cada usuario.

b) **Modificar una columna existente**: el comando ALTER COLUMN o MODIFY permite cambiar las características de una columna, como el tipo de dato o su tamaño. Por ejemplo:

```
ALTER TABLE Usuario

ALTER COLUMN nombre VARCHAR(150);
```

En este ejemplo se modifica la longitud del campo nombre, ampliando su capacidad para almacenar más caracteres.

c) **Eliminar una columna**: el comando DROP COLUMN permite eliminar una columna de una tabla cuando ya no es necesaria. Por ejemplo:

```
ALTER TABLE Usuario

DROP COLUMN telefono;
```

Esta instrucción elimina la columna telefono de la tabla Usuario.

La modificación de estructuras de tablas y columnas permite adaptar la base de datos a las necesidades cambiantes de los sistemas de información, manteniendo la organización y consistencia de los datos almacenados.

4.4. Inserción, actualización y eliminación de registros

La gestión de la información almacenada en una base de datos implica realizar diferentes operaciones sobre los registros de las tablas. Estas operaciones permiten agregar nuevos datos, modificar información existente o eliminar registros que ya no son necesarios. Para ello se utilizan los comandos del Data Manipulation Language (DML) del Structured Query Language (SQL).

Estas acciones permiten mantener actualizada la información dentro del sistema y garantizan que los datos reflejen correctamente las actividades o procesos que se registran en la base de datos.

a) **Inserción de registros**: la inserción de datos se realiza mediante el comando INSERT, el cual permite agregar nuevos registros a una tabla. Por ejemplo:

```
INSERT INTO Usuario (id_usuario, nombre, correo)
VALUES (1, 'Ana López', 'ana@correo.com');
```

En este caso se agrega un nuevo registro a la tabla Usuario, donde se almacenan los datos correspondientes a un usuario del sistema.

b) **Actualización de registros**: la actualización de datos se realiza mediante el comando UPDATE, el cual permite modificar la información de uno o varios registros existentes. Por ejemplo:

```
UPDATE Usuario
SET correo = 'ana.lopez@correo.com'
WHERE id_usuario = 1;
```

En este ejemplo se actualiza el correo electrónico del usuario cuyo identificador es 1.

c) **Eliminación de registros**: se realiza mediante el comando DELETE, el cual permite borrar información almacenada en una tabla. Por ejemplo:

```
DELETE FROM Usuario
WHERE id_usuario = 1;
```

En este caso se elimina el registro del usuario con identificador 1 de la tabla Usuario.

Estas operaciones permiten manipular la información almacenada en la base de datos de manera controlada, facilitando la administración, actualización y mantenimiento de los datos dentro de los sistemas de información.

4.5. Eliminación de tablas y bases de datos

En la administración de bases de datos también es posible eliminar estructuras que ya no son necesarias dentro del sistema. Estas operaciones se realizan mediante

comandos del Data Definition Language (DDL) del Structured Query Language (SQL), los cuales permiten eliminar tablas completas o incluso bases de datos.

La eliminación de estos elementos debe realizarse con precaución, ya que al ejecutar estos comandos se pierde la información almacenada y la estructura asociada.

a) **Eliminación de tablas:** para eliminar una tabla de una base de datos se utiliza el comando DROP TABLE. Este comando elimina tanto la estructura de la tabla como todos los registros que contiene. Por ejemplo:

```
DROP TABLE Usuario;
```

En este caso se elimina completamente la tabla Usuario, junto con todos los datos almacenados en ella.

b) **Eliminación de bases de datos**: cuando se requiere eliminar una base de datos completa, se utiliza el comando DROP DATABASE. Esta instrucción elimina la base de datos y todas las tablas y estructuras que contiene. Por ejemplo:

```
DROP DATABASE Biblioteca;
```

En este ejemplo se elimina la base de datos llamada Biblioteca, incluyendo todas las tablas y registros asociados.

La eliminación de tablas y bases de datos forma parte de las tareas de mantenimiento y administración de los sistemas de información. Sin embargo, estas acciones deben ejecutarse con cuidado para evitar la pérdida accidental de datos importantes.

5. Gestión y administración de bases de datos

La gestión y administración de bases de datos comprende el conjunto de actividades orientadas a garantizar el correcto funcionamiento, organización y mantenimiento de la información almacenada en un sistema. Estas tareas incluyen la creación de bases de datos, la configuración de los sistemas gestores, la supervisión del rendimiento, la seguridad de la información y el control del acceso a los datos.

En los sistemas de información actuales, las bases de datos se gestionan mediante software especializado conocido como Database Management System (DBMS). Estos sistemas permiten crear, almacenar, consultar y administrar grandes volúmenes de información de manera organizada y eficiente.

La administración adecuada de una base de datos permite asegurar la integridad, disponibilidad y consistencia de la información. Además, facilita la realización de copias de seguridad, la recuperación de datos ante fallos del sistema y la optimización del rendimiento de las consultas.

Dentro de este proceso también se utilizan herramientas de modelado y administración que permiten diseñar la estructura de las bases de datos, visualizar sus relaciones y gestionar su funcionamiento en diferentes entornos tecnológicos.

5.1. Sistemas gestores de bases de datos

Un Database Management System (DBMS) es un software especializado que permite crear, almacenar, organizar, consultar y administrar la información contenida en una base de datos. Su función principal es facilitar el manejo de grandes volúmenes de datos de forma estructurada, garantizando su integridad, seguridad y disponibilidad.

Los sistemas gestores de bases de datos actúan como intermediarios entre los usuarios o aplicaciones y la base de datos, permitiendo realizar diferentes operaciones como la creación de tablas, la inserción de registros, la consulta de información y la actualización o eliminación de datos.

Entre las principales funciones de un sistema gestor de bases de datos se encuentran:

- Administrar el almacenamiento y organización de los datos.
- Permitir el acceso y la manipulación de la información mediante lenguajes de consulta como SQL.
- Garantizar la integridad y consistencia de los datos.
- Controlar el acceso de los usuarios mediante mecanismos de seguridad.
- Facilitar la realización de copias de seguridad y la recuperación de información.

Actualmente existen diferentes tipos de sistemas gestores de bases de datos que se utilizan según las necesidades del sistema de información. Algunos trabajan con modelos relacionales, mientras que otros utilizan modelos NoSQL orientados a manejar grandes volúmenes de datos o estructuras de información más flexibles.

Entre los sistemas gestores de bases de datos más utilizados se encuentran MySQL, Oracle, PostgreSQL, MongoDB, Cassandra y Neo4j, los cuales ofrecen diferentes características y aplicaciones dentro del desarrollo de sistemas de información.

5.2. Configuración básica de sistemas gestores de bases de datos

La configuración inicial de un sistema gestor de bases de datos consiste en preparar el entorno necesario para crear, administrar y consultar bases de datos. Este proceso generalmente incluye la instalación del software, la creación de usuarios, la configuración de parámetros básicos y la verificación del funcionamiento del sistema.

Dependiendo del tipo de base de datos que se utilice, los procedimientos de configuración pueden variar. Algunos gestores están diseñados para trabajar con bases de datos relacionales, mientras que otros utilizan modelos NoSQL, orientados a manejar grandes volúmenes de datos o estructuras más flexibles.

Entre los sistemas gestores de bases de datos más utilizados se encuentran MySQL, Oracle y PostgreSQL para bases de datos relacionales, y MongoDB, Cassandra y Neo4j en el contexto de bases de datos NoSQL.

A continuación, se presenta una comparación general de estos gestores y sus características principales:

Tabla 15. Sistemas gestores de bases de datos y sus características generales

Sistema gestor	Tipo de base de datos	Característica principal
MySQL.	Relacional.	Amplio uso en aplicaciones web y facilidad de administración.
Oracle.	Relacional.	Alta capacidad para entornos empresariales y grandes volúmenes de datos.
PostgreSQL.	Relacional.	Sistema robusto de código abierto con alta compatibilidad con estándares SQL.
MongoDB.	NoSQL (documentos).	Almacena información en documentos tipo JSON.

Sistema gestor	Tipo de base de datos	Característica principal
Cassandra.	NoSQL (columnas).	Diseñado para manejar grandes volúmenes de datos distribuidos.
Neo4j.	NoSQL (grafos).	Especializado en el manejo de relaciones complejas entre datos.

Esta clasificación permite comprender que los sistemas gestores de bases de datos se adaptan a diferentes necesidades tecnológicas, dependiendo del tipo de información que se requiera almacenar y procesar.

5.3. Herramientas de modelado de bases de datos

Las herramientas de modelado de bases de datos son aplicaciones que permiten diseñar, visualizar y estructurar bases de datos antes de su implementación en un sistema gestor de bases de datos. Estas herramientas facilitan la creación de diagramas, la definición de entidades, atributos y relaciones, así como la generación automática de scripts que pueden utilizarse para crear la base de datos en diferentes plataformas.

El uso de herramientas de modelado permite organizar de manera clara la estructura de la información, identificar posibles errores en el diseño y documentar el funcionamiento del sistema de datos. De esta manera, los desarrolladores y analistas pueden comprender con mayor facilidad cómo se relacionan los diferentes elementos dentro de la base de datos.

Entre las funciones más comunes de estas herramientas se encuentran:

- Creación de diagramas Entidad–Relación.

- Diseño de modelos relacionales.
- Generación automática de código SQL.
- Documentación del modelo de datos.
- Integración con diferentes sistemas gestores de bases de datos.

Existen diversas herramientas utilizadas en el diseño de bases de datos, cada una con características y funcionalidades específicas:

Tabla 16. Herramientas de modelado de bases de datos

Herramienta	Característica principal
Oracle Data Modeler.	Permite diseñar modelos conceptuales, lógicos y físicos en entornos basados en Oracle.
MySQL Workbench.	Facilita el modelado visual y la administración de bases de datos MySQL.
PowerDesigner.	Herramienta empresarial utilizada para modelado avanzado de datos.
Visual Paradigm.	Permite modelar sistemas utilizando diferentes tipos de diagramas, incluido el modelo ER.
AWS NoSQL Workbench.	Herramienta para diseñar y visualizar bases de datos NoSQL en entornos de AWS.

Estas herramientas permiten planificar y estructurar adecuadamente las bases de datos, facilitando su implementación, mantenimiento y evolución dentro de los sistemas de información.

6. Procesamiento y almacenamiento de datos

El procesamiento y almacenamiento de datos constituye un aspecto fundamental en el funcionamiento de los sistemas de información, ya que permite gestionar grandes volúmenes de datos de manera organizada, segura y eficiente. En una base de datos, el procesamiento de la información se refiere a las operaciones que se realizan sobre los datos, como consultas, inserciones, actualizaciones y análisis, mientras que el almacenamiento se relaciona con la forma en que los datos se guardan y estructuran dentro del sistema.

Los sistemas de bases de datos están diseñados para soportar diferentes tipos de cargas de trabajo, dependiendo del tipo de operaciones que se realizan con la información. Algunas aplicaciones requieren gestionar transacciones en tiempo real, como ocurre en sistemas bancarios o de comercio electrónico, mientras que otras se enfocan en el análisis de grandes volúmenes de datos para la toma de decisiones.

Para garantizar la confiabilidad de la información, las bases de datos implementan mecanismos que aseguran que las transacciones se ejecuten de forma correcta y consistente. Estos mecanismos permiten mantener la integridad de los datos incluso cuando múltiples usuarios acceden al sistema al mismo tiempo o cuando se presentan fallos en el proceso.

Además, el almacenamiento de datos puede adoptar diferentes estructuras y tecnologías dependiendo de las necesidades del sistema. Existen modelos diseñados para manejar información estructurada, así como soluciones orientadas al almacenamiento masivo de datos y análisis avanzado.

Comprender cómo se procesan y almacenan los datos permite diseñar sistemas de información más eficientes, optimizar el rendimiento de las bases de datos y garantizar la disponibilidad y confiabilidad de la información dentro de las organizaciones.

6.1. Cargas de trabajo transaccionales

Las cargas de trabajo transaccionales corresponden a las operaciones que se realizan de manera continua sobre una base de datos para registrar, consultar o modificar información en tiempo real. Este tipo de procesamiento se caracteriza por manejar un gran número de transacciones pequeñas que deben ejecutarse de forma rápida, precisa y segura.

En los sistemas transaccionales, cada operación representa una transacción que modifica o consulta datos dentro de la base de datos. Estas transacciones suelen involucrar acciones como insertar nuevos registros, actualizar información existente o consultar datos específicos. Debido a que muchas aplicaciones requieren acceso simultáneo por parte de múltiples usuarios, el sistema gestor de bases de datos debe garantizar que cada transacción se ejecute correctamente sin afectar la consistencia de la información.

Las cargas de trabajo transaccionales son comunes en sistemas operativos de uso diario dentro de las organizaciones. Algunos ejemplos incluyen sistemas bancarios, plataformas de comercio electrónico, sistemas de matrícula académica y aplicaciones de gestión empresarial, donde cada operación realizada por los usuarios debe registrarse inmediatamente en la base de datos.

Este tipo de procesamiento se asocia generalmente con sistemas conocidos como OLTP (Online Transaction Processing), los cuales están diseñados para gestionar grandes volúmenes de transacciones simultáneas manteniendo la integridad y disponibilidad de los datos. Estas características permiten que las bases de datos soporten operaciones críticas que requieren rapidez, confiabilidad y control en el manejo de la información.

6.2. Cargas de trabajo analíticas

Las cargas de trabajo analíticas se refieren al procesamiento y análisis de grandes volúmenes de datos con el propósito de obtener información útil para la toma de decisiones. A diferencia de las cargas transaccionales, que se centran en registrar y gestionar operaciones en tiempo real, las cargas analíticas se orientan al estudio, exploración e interpretación de datos históricos o acumulados.

Este tipo de cargas permite identificar patrones, tendencias y relaciones entre los datos, lo cual resulta fundamental para procesos de análisis empresarial, generación de reportes, inteligencia de negocios y apoyo a la planificación estratégica. Las consultas analíticas suelen ser más complejas, ya que implican la agregación, filtrado y comparación de grandes conjuntos de información.

Las cargas de trabajo analíticas se utilizan comúnmente en sistemas como almacenes de datos (data warehouse) y plataformas de análisis que facilitan la elaboración de informes, cuadros de mando y visualizaciones que ayudan a comprender el comportamiento de los datos en diferentes contextos organizacionales. De esta manera, contribuyen a transformar los datos en conocimiento útil para mejorar procesos y apoyar la toma de decisiones informadas.

6.3. Principios ACID y gestión de transacciones

Los principios ACID constituyen un conjunto de propiedades que garantizan la fiabilidad y consistencia de las transacciones en los sistemas de bases de datos. Una transacción corresponde a una secuencia de operaciones que se ejecutan como una única unidad de trabajo, asegurando que los datos se mantengan correctos incluso ante fallos del sistema, errores o interrupciones.

El término ACID proviene de cuatro propiedades fundamentales que permiten mantener la integridad de la información durante la ejecución de las transacciones:

a) Atomicidad

Garantiza que todas las operaciones de una transacción se ejecuten completamente o, en caso de presentarse un error, ninguna se aplique. Esto evita que la base de datos quede en un estado intermedio o inconsistente.

b) Consistencia

Asegura que la base de datos pase de un estado válido a otro, respetando las reglas, restricciones y relaciones definidas en el sistema.

c) Aislamiento

Permite que varias transacciones se ejecuten simultáneamente sin interferir entre sí, evitando que los resultados parciales de una transacción afecten a otra.

d) Durabilidad

Establece que, una vez confirmada una transacción, los cambios realizados se almacenan de manera permanente, incluso si ocurre una falla en el sistema.

La gestión de transacciones es el mecanismo mediante el cual el sistema gestor de bases de datos controla la ejecución de estas operaciones, incluyendo procesos como el inicio, la confirmación o la cancelación de una transacción. De esta forma, se garantiza la integridad de los datos y el funcionamiento confiable de las aplicaciones que dependen del manejo de la información.

6.4. Tipos de almacenamiento de datos

El almacenamiento de datos hace referencia a las diferentes formas en que la información se guarda físicamente o de manera lógica dentro de los sistemas informáticos. En los entornos de bases de datos, el tipo de almacenamiento influye en la velocidad de acceso a la información, la capacidad de procesamiento, la escalabilidad del sistema y la seguridad de los datos.

Dependiendo de la arquitectura del sistema y de las necesidades de procesamiento, es posible identificar diferentes tipos de almacenamiento de datos:

a) Almacenamiento Local

Los datos se guardan en servidores o equipos físicos dentro de la infraestructura de una organización. Este modelo permite un control directo sobre los recursos y la seguridad de la información.

b) Almacenamiento en la nube

Los datos se almacenan en infraestructuras distribuidas gestionadas por proveedores de servicios en la nube. Este tipo de almacenamiento facilita la escalabilidad, el acceso remoto y la disponibilidad de la información.

c) Almacenamiento distribuido

La información se almacena en múltiples servidores o nodos conectados entre sí. Este enfoque permite mejorar el rendimiento, la tolerancia a fallos y la capacidad de manejar grandes volúmenes de datos.

d) Almacenamiento orientado a bases de datos

Los datos se organizan y almacenan dentro de sistemas gestores de bases de datos, los cuales permiten administrar la información mediante esquemas, consultas y mecanismos de control que garantizan la integridad y consistencia de los datos.

La comprensión de estos tipos de almacenamiento permite seleccionar la arquitectura tecnológica más adecuada para gestionar la información de manera eficiente, segura y escalable dentro de los sistemas de información.

7. Bases de datos NoSQL y sistemas distribuidos

Las bases de datos NoSQL surgen como una alternativa a los sistemas relacionales tradicionales para gestionar grandes volúmenes de información, especialmente en entornos donde los datos son variados, crecen rápidamente y requieren alta disponibilidad. A diferencia de las bases de datos relacionales, que utilizan tablas estructuradas y esquemas rígidos, las bases de datos NoSQL permiten almacenar información de forma más flexible, adaptándose a distintos tipos de datos como documentos, grafos, claves-valor o columnas.

Este tipo de bases de datos se utiliza ampliamente en aplicaciones web, plataformas de comercio electrónico, redes sociales, sistemas de análisis de datos y servicios en la nube, donde la escalabilidad y el rendimiento son factores fundamentales. Además, muchas de estas tecnologías están diseñadas para funcionar en arquitecturas distribuidas, lo que significa que los datos pueden almacenarse y procesarse en múltiples servidores de forma simultánea.

En este contexto, los sistemas distribuidos permiten mejorar la disponibilidad del servicio, aumentar la capacidad de procesamiento y garantizar que la información continúe accesible incluso ante fallos en alguno de los nodos del sistema. Para lograrlo, se emplean mecanismos de replicación, particionamiento de datos y balanceo de carga.

La comprensión de las bases de datos NoSQL y de los principios de los sistemas distribuidos resulta fundamental en el desarrollo de aplicaciones modernas, ya que permiten gestionar grandes cantidades de información de manera eficiente, flexible y escalable en entornos tecnológicos cada vez más complejos.

7.1. Concepto y tipos de bases de datos NoSQL

Las bases de datos NoSQL corresponden a un conjunto de sistemas de gestión de datos diseñados para almacenar y procesar grandes volúmenes de información de manera flexible y escalable. A diferencia de las bases de datos relacionales, estas no utilizan necesariamente tablas con esquemas rígidos, lo que permite trabajar con datos estructurados, semiestructurados o desestructurados.

El término NoSQL significa “Not Only SQL” (no solo SQL), lo que indica que estos sistemas no buscan reemplazar completamente a las bases de datos relacionales, sino ofrecer alternativas que se adapten mejor a determinados tipos de aplicaciones, especialmente aquellas que requieren alto rendimiento, distribución de datos y capacidad de escalar en múltiples servidores.

Las bases de datos NoSQL se clasifican principalmente en los siguientes tipos:

a) Bases de datos clave-valor

Almacenan la información en pares formados por una clave y un valor asociado. Este modelo es sencillo y permite realizar consultas muy rápidas. Es utilizado en sistemas de caché, sesiones de usuario y aplicaciones que requieren acceso rápido a los datos.

b) Bases de datos orientadas a documentos

Guardan la información en documentos estructurados generalmente en formatos como JSON o BSON. Cada documento puede contener diferentes atributos, lo que brinda mayor flexibilidad para manejar información diversa.

c) Bases de datos orientadas a columnas

Organizan la información por columnas en lugar de filas. Este modelo es eficiente para procesar grandes cantidades de datos y realizar análisis en sistemas que manejan grandes volúmenes de información.

d) Bases de datos orientadas a grafos

Están diseñadas para representar relaciones complejas entre los datos mediante nodos y conexiones. Son utilizadas en redes sociales, sistemas de recomendación y análisis de relaciones entre entidades.

La elección del tipo de base de datos NoSQL depende de las necesidades del sistema, del tipo de información que se desea almacenar y de los requisitos de rendimiento y escalabilidad de la aplicación.

7.2. Operaciones y manipulación de datos en bases de datos NoSQL

Las bases de datos NoSQL permiten realizar diversas operaciones para gestionar la información almacenada dentro del sistema. Estas operaciones permiten crear, consultar, modificar y eliminar datos, de forma similar a lo que ocurre en las bases de datos relacionales, aunque con estructuras y comandos adaptados a cada tipo de base de datos.

En los sistemas NoSQL, la manipulación de datos se realiza generalmente mediante consultas específicas del gestor de base de datos o a través de interfaces de programación que permiten interactuar con la información almacenada. Estas

operaciones son fundamentales para garantizar que los datos puedan ser utilizados, actualizados y gestionados de manera eficiente dentro de las aplicaciones.

Las principales operaciones de manipulación de datos en bases de datos NoSQL incluyen las siguientes:

a) Inserción de datos

Permite agregar nuevos registros o documentos dentro de la base de datos. Dependiendo del tipo de base de datos, la información puede almacenarse como documentos, pares clave-valor o nodos en un grafo.

b) Consulta de datos

Permite recuperar información almacenada en la base de datos mediante diferentes criterios de búsqueda. Las consultas pueden realizarse por identificadores, atributos específicos o condiciones definidas por el usuario.

c) Actualización de datos

Permite modificar la información existente dentro de los registros o documentos almacenados en la base de datos, manteniendo la integridad y consistencia de los datos.

d) Eliminación de datos

Permite borrar registros, documentos o elementos que ya no son necesarios dentro del sistema.

Estas operaciones constituyen la base para la manipulación de datos en sistemas NoSQL y permiten que las aplicaciones puedan gestionar grandes volúmenes de información de manera flexible y eficiente.

7.3. Consultas, filtrado y agregación de datos

En las bases de datos NoSQL, las consultas permiten recuperar información almacenada en colecciones, documentos o registros según diferentes criterios. A diferencia de las bases de datos relacionales, donde las consultas se realizan principalmente mediante el lenguaje SQL, en los sistemas NoSQL las consultas se ejecutan utilizando estructuras propias de cada gestor de base de datos.

Las consultas permiten seleccionar información específica dentro de la base de datos, facilitando el acceso a los datos que cumplen determinadas condiciones. Para lograrlo, se utilizan diferentes mecanismos como el filtrado y la agregación de datos.

Entre las operaciones más utilizadas se encuentran las siguientes:

a) **Consultas de datos**: permiten recuperar documentos o registros almacenados dentro de una colección. Por ejemplo, en una base de datos que almacena información de estudiantes, se puede consultar toda la colección de estudiantes para visualizar los registros existentes. Por ejemplo:

   db.estudiantes.find()

b) **Filtrado de información**: permite seleccionar únicamente los registros que cumplen ciertas condiciones dentro de la consulta. Esto facilita encontrar datos específicos dentro de grandes volúmenes de información. Por ejemplo:

   db.estudiantes.find({edad: 20})

   Esta consulta devuelve los estudiantes cuya edad es igual a 20.

c) **Agregación de datos**: permite realizar operaciones de cálculo o agrupación sobre un conjunto de datos, como contar registros, calcular promedios o agrupar información según determinados atributos. Por ejemplo:

```
db.estudiantes.aggregate([
 { $group: { _id: "$programa", total: { $sum: 1 } } }
])
```

Este ejemplo permite contar cuántos estudiantes existen en cada programa académico.

En conjunto, estas operaciones permiten consultar, organizar y analizar grandes volúmenes de información dentro de las bases de datos NoSQL, facilitando su uso en aplicaciones modernas que requieren alta escalabilidad y flexibilidad en el manejo de datos.

7.4. Teorema CAP y sistemas distribuidos

En los sistemas de bases de datos distribuidos, la información se almacena y procesa en varios servidores o nodos que trabajan de manera conjunta. Este tipo de arquitectura permite manejar grandes volúmenes de datos y garantizar una mayor disponibilidad del sistema. Sin embargo, también implica enfrentar desafíos relacionados con la consistencia y la disponibilidad de la información.

En este contexto surge el Teorema CAP, un principio que explica las limitaciones que existen en los sistemas distribuidos. Este teorema establece que un sistema

distribuido no puede garantizar simultáneamente tres propiedades fundamentales cuando ocurre una partición de red.

Estas propiedades son las siguientes:

- **Consistencia (Consistency)**: todos los nodos del sistema muestran la misma información al mismo tiempo. Cuando se realiza una actualización de datos, cualquier usuario que consulte el sistema obtiene el mismo resultado.
- **Disponibilidad (Availability)**: el sistema responde a las solicitudes de los usuarios incluso si algunos nodos presentan fallas. Esto significa que el servicio permanece accesible.
- **Tolerancia a particiones (Partition Tolerance)**: el sistema continúa funcionando incluso cuando ocurre una falla de comunicación entre algunos nodos de la red.

El Teorema CAP indica que, en presencia de una partición de red, un sistema solo puede garantizar dos de estas tres propiedades al mismo tiempo. Por esta razón, los diseñadores de sistemas deben priorizar cuáles características son más importantes según las necesidades de la aplicación.

Por ejemplo, algunos sistemas priorizan la consistencia y la tolerancia a particiones (CP), mientras que otros priorizan la disponibilidad y la tolerancia a particiones (AP). Esta decisión depende del tipo de aplicación y del comportamiento esperado del sistema.

Comprender el Teorema CAP permite analizar cómo funcionan los sistemas distribuidos y cómo las bases de datos modernas equilibran la consistencia, la

disponibilidad y la tolerancia a fallos para garantizar el funcionamiento adecuado de las aplicaciones.

En la práctica, el Teorema CAP se utiliza como una guía para decidir qué propiedades deben priorizarse en un sistema distribuido. El proceso de análisis generalmente sigue estas etapas:

a) Identificar el tipo de aplicación: se analiza si el sistema requiere mayor consistencia de los datos o mayor disponibilidad del servicio.

b) Evaluar los posibles fallos de red: en los sistemas distribuidos es común que existan interrupciones en la comunicación entre nodos, por lo que la tolerancia a particiones suele ser una propiedad necesaria.

c) Definir las prioridades del sistema: dependiendo del tipo de aplicación, se priorizan dos de las tres propiedades del teorema CAP. Entre ella se encuentran:

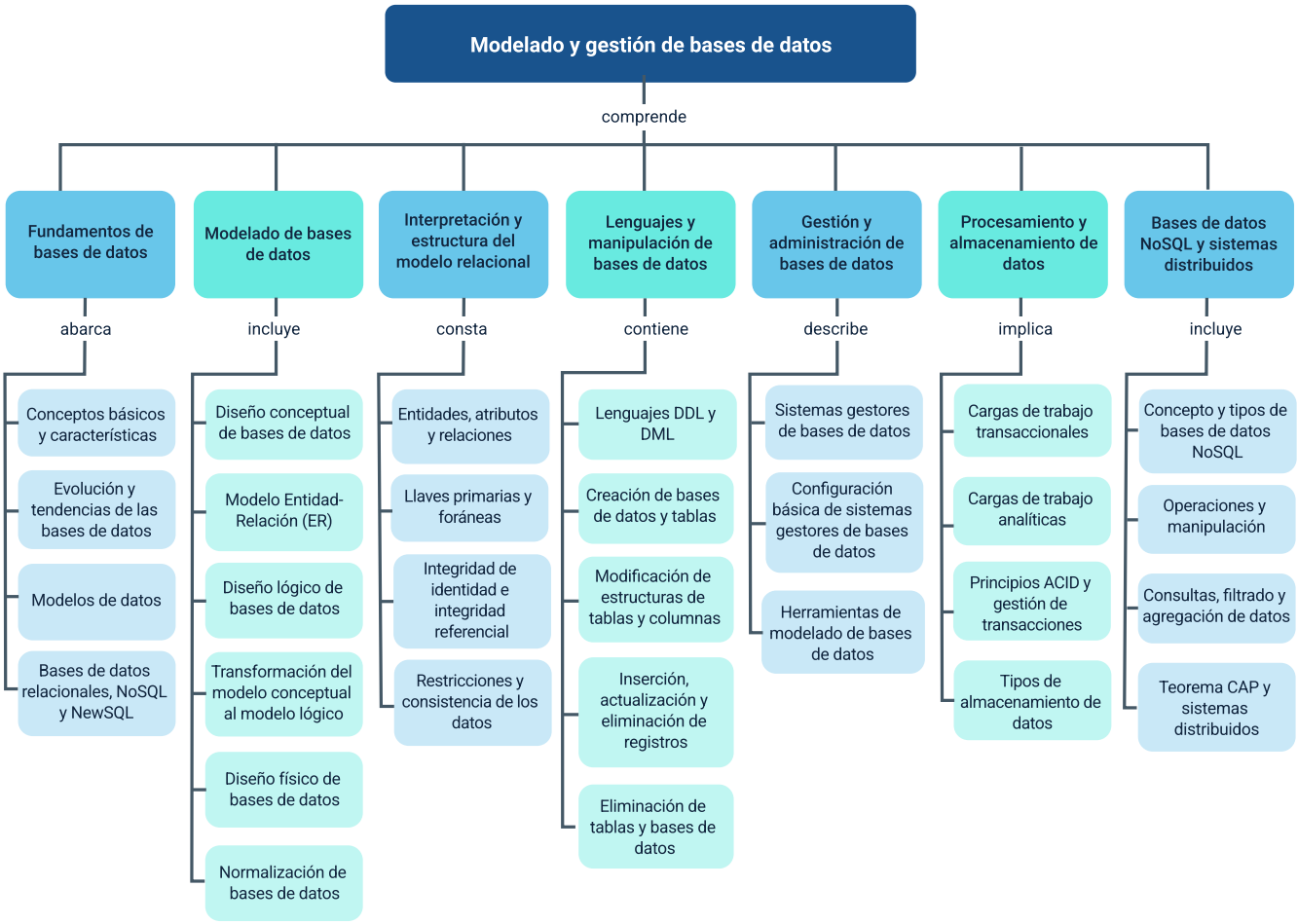
- Consistencia y tolerancia a particiones (CP).
- Disponibilidad y tolerancia a particiones (AP).

d) Seleccionar la tecnología adecuada: a partir de estas decisiones se eligen las bases de datos o arquitecturas que mejor se adapten a los requerimientos del sistema.

De esta manera, el Teorema CAP se convierte en una herramienta conceptual para comprender el comportamiento de los sistemas distribuidos y orientar el diseño de infraestructuras de datos modernas.

Síntesis

Las bases de datos surgen como una solución tecnológica para organizar y gestionar grandes volúmenes de información que anteriormente se almacenaban en registros físicos. A través de diferentes modelos y procesos de diseño, los datos se estructuran en tablas y relaciones que permiten garantizar su organización e integridad dentro de los sistemas gestores de bases de datos. Posteriormente, mediante lenguajes especializados, es posible crear, modificar, consultar y eliminar información según las necesidades de cada aplicación. En la actualidad, las bases de datos no solo almacenan información, sino que también permiten procesarla, analizarla y distribuirla en sistemas más complejos, incluyendo arquitecturas distribuidas y tecnologías NoSQL, lo que facilita la gestión eficiente de los datos y su uso para la toma de decisiones en diversos entornos digitales.


Modelado y gestión de bases de datos
comprende
Fundamentos de bases de datos
Modelado de bases de datos
Interpretación y estructura del modelo relacional
Lenguajes y manipulación de bases de datos
Gestión y administración de bases de datos
Procesamiento y almacenamiento de datos
Bases de datos NoSQL y sistemas distribuidos
abarca
incluye
consta
contiene
describe
implica
incluye
Conceptos básicos y características
Evolución y tendencias de las bases de datos
Modelos de datos
Bases de datos relacionales, NoSQL y NewSQL
Diseño conceptual de bases de datos
Modelo Entidad-Relación (ER)
Diseño lógico de bases de datos
Transformación del modelo conceptual al modelo lógico
Diseño físico de bases de datos
Normalización de bases de datos
Entidades, atributos y relaciones
Llaves primarias y foráneas
Integridad de identidad e integridad referencial
Restricciones y consistencia de los datos
Lenguajes DDL y DML
Creación de bases de datos y tablas
Modificación de estructuras de tablas y columnas
Inserción, actualización y eliminación de registros
Eliminación de tablas y bases de datos
Sistemas gestores de bases de datos
Configuración básica de sistemas gestores de bases de datos
Herramientas de modelado de bases de datos
Cargas de trabajo transaccionales
Cargas de trabajo analíticas
Principios ACID y gestión de transacciones
Tipos de almacenamiento de datos
Concepto y tipos de bases de datos NoSQL
Operaciones y manipulación
Consultas, filtrado y agregación de datos
Teorema CAP y sistemas distribuidos

Glosario

Agregación: proceso que permite realizar operaciones sobre un conjunto de datos para obtener resultados resumidos, como conteos, promedios, sumas o valores máximos y mínimos dentro de una base de datos.

Base de datos: conjunto organizado de datos relacionados entre sí que se almacenan y gestionan de forma estructurada para facilitar su consulta, modificación y análisis.

Consistencia: propiedad que garantiza que todos los nodos de un sistema distribuido muestran los mismos datos al mismo tiempo después de realizar una operación.

Disponibilidad: capacidad de un sistema para responder a las solicitudes de los usuarios en todo momento, incluso si algunos componentes presentan fallas.

Filtrado de datos: proceso que permite seleccionar únicamente los registros que cumplen ciertas condiciones dentro de una consulta en una base de datos.

Integridad de datos: conjunto de reglas y mecanismos que aseguran que la información almacenada en una base de datos sea precisa, coherente y confiable.

Modelo de datos: representación conceptual que define cómo se organizan, relacionan y estructuran los datos dentro de una base de datos.

Modelo relacional: modelo de base de datos que organiza la información en tablas compuestas por filas y columnas, estableciendo relaciones entre ellas mediante claves.

NoSQL: tipo de bases de datos no relacionales diseñadas para manejar grandes volúmenes de datos y estructuras flexibles, comúnmente utilizadas en aplicaciones web y sistemas distribuidos.

Tabla: estructura fundamental del modelo relacional donde se almacenan los datos organizados en filas (registros) y columnas (atributos).

Teorema CAP: principio de los sistemas distribuidos que establece que un sistema no puede garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones en una red distribuida.

Tolerancia a particiones: capacidad de un sistema distribuido para continuar funcionando incluso cuando se presentan fallas de comunicación entre algunos nodos de la red.

Referencias bibliográficas

Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9–36. https://doi.org/10.1145/320434.320440

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Coronel, C., & Morris, S. (2015). Database systems: Design, implementation, and management (11th ed.). Cengage Learning.

Date, C. J. (2004). Introducción a los sistemas de bases de datos (8.ª ed.). Pearson Educación.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.

Ecosistema de Recursos Educativos Digitales SENA. (2023). Bases de datos NoSQL. YouTube. https://www.youtube.com/watch?v=M-Iw1awJ1NU

Ecosistema de Recursos Educativos Digitales SENA. (2024). Construcción de un modelo entidad relación. YouTube. https://www.youtube.com/watch?v=t7KZFiCstwI

Garcia-Molina, H., Ullman, J. D., & Widom, J. (2009). Database systems: The complete book (2nd ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill Education.

Stonebraker, M., & Hellerstein, J. M. (2005). What goes around comes around. Readings in Database Systems (4th ed.). MIT Press.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Veimar Celis Melendez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluador ade contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima